



PROGRAMMATION FONCTIONNELLE
via le langage Haskell

A. OUALI

L2 au département Mathématique-Informatique
UPR des sciences
Université de Caen Normandie



Système de typage type



- Il est possible de paramétrer la définition du nouveau type

```
type Ensemble a = [a]
```

- Types avec plus d'un paramètre sont également possibles.

```
type Assoc k v = [(k , v )]  
  
find :: Eq k -> k -> Assoc k v -> v  
find k t = head [v | (k', v ) <- t, k == k']
```

A. Ouali

Programmation fonctionnelle en Haskell

2 / 20

Exercice fonction



- Nous considérons dans cet exercice l'addition et la multiplication sur les matrices. Pour représenter les matrices dans Haskell, nous utilisons une liste de listes d'entiers. Par exemple :

La matrice : $\begin{pmatrix} 1 & 4 & 9 \\ 2 & 5 & 7 \end{pmatrix} \rightarrow \begin{matrix} [[1, 4, 9], \\ [2, 5, 7]] \end{matrix}$
- Définir un type appelé `Matrice` pour représenter les matrices comme une liste de listes d'entiers. Notre première tâche est d'écrire un test pour vérifier si cette représentation nous donne est une matrice. Ce test doit vérifier deux conditions :
 - que les listes d'entiers sont toutes de longueur égale,
 - qu'il y a au moins une ligne et une colonne pour avoir une matrice.
- Définir une fonction `uniforme :: [Int] -> Bool` qui teste si les entiers d'une liste sont tous égaux. Vous pouvez utiliser la fonction d'ordre supérieur `all`. Proposer une autre définition équivalente en utilisant `foldr` et `map`.
- En utilisant votre fonction `uniforme`, définir une fonction `valide :: Matrix -> Bool` qui teste si une liste de listes d'entiers est une matrice (elle devrait tester les propriétés 1) et 2) spécifiées ci-dessus).

A. Ouali

Programmation fonctionnelle en Haskell

3 / 20

Exercice fonction



- L'addition de deux matrices de taille égale se fait en ajoutant par paires les éléments qui sont dans la même position, par exemple :

 $\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} + \begin{pmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \end{pmatrix} = \begin{pmatrix} 11 & 22 & 33 \\ 44 & 55 & 66 \end{pmatrix}$
- Définir une fonction `plusM` qui additionne deux matrices. Renvoie une erreur si l'entrée ne convient pas. Il peut être utile de définir d'autres fonctions pour définir la fonction.

A. Ouali

Programmation fonctionnelle en Haskell

5 / 20

Exercice fonction



- Une autre fonction utile pour travailler avec des paires est `uncurry`, qui transforme une fonction qui prend deux arguments en une fonction qui opère sur une paire.


```
Main> uncurry (+) (10,8)  
18
```
- Une fonction d'ordre supérieur utile est `zipWith`. Elle ressemble beaucoup à la fonction `zip` ou `myZip`. La différence est qu'au lieu de combiner des éléments par paire, vous pouvez donner à `zipWith` une fonction spécifique pour combiner chacun des deux éléments.


```
*Main> zipWith (*) [1,2,3] [4,5,6]  
[4,10,18]
```
- Définir `myZipWith` en utilisant `zip` et les fonctions d'ordre supérieur `map` et `uncurry`.

A. Ouali

Programmation fonctionnelle en Haskell

4 / 20